

Andromeda SGBDRT and SRPL master architecture

A consolidated design reference for a modern
transactional relational database system in 2026

Scope: Rust, QUIC, Protobuf contracts without gRPC, SRPL, transactional kernel, WAL/MVCC/recovery, hot/cold storage, catalog, security, observability, HA/DR, and roadmap.

Style: American English, Microsoft Documentation-inspired structure, source-tagged, decision-oriented.

Consolidation draft 1.0

May 3, 2026

Prepared from the project source corpus and research references available in the workspace.

Contents

[Executive summary](#)

[Document scope and evidence model](#)

[Consolidated source ledger](#)

[Design doctrine](#)

[System overview](#)

[Functional engine architecture](#)

[Rust implementation baseline](#)

[QUIC plus Protobuf without gRPC](#)

[Procedure contract design](#)

[SRPL language model](#)

[Type system and data model](#)

[Catalog and metadata architecture](#)

[Transaction kernel](#)

[Storage architecture](#)

[Recovery and forensic startup](#)

[Maps and internal analytics](#)

[Procedure Store, optimizer, statistics, and evidence](#)

[Security, IAM, administration, and audit](#)

[HA/DR, backup, restore, and forensic operations](#)

[Observability and operability](#)

[Hardware acceleration boundaries](#)

[External research foundation](#)

[V0 roadmap](#)

[Open decisions and decision debt](#)

[Native vocabulary and naming discipline](#)

[Wire protocol deep specification](#)

[Protobuf schema governance](#)

[Typed client and C# source generator strategy](#)

[SRPL compiler and diagnostics](#)

[Storage binary specification backlog](#)

[Testing, verification, and benchmark discipline](#)

[Operational profiles and resource governance](#)

[Error model and completion semantics](#)

[Normalized project source transcriptions](#)

[Appendix A - Decision register](#)

[Appendix B - Risk register](#)

[Appendix C - Source-to-section traceability](#)

[Appendix D - External reference ledger](#)

[Appendix E - Canonical glossary](#)

[Appendix F - Protobuf contract baseline](#)

[Appendix G - Minimal V0 Protobuf envelope sketch](#)

[Appendix H - Minimal SRPL grammar direction](#)

Use the PDF outline pane for navigation.

1. Executive summary

Andromeda is a modern transactional relational database system, referred to in this document as an **SGBDRT**: a relational, transactional, typed, deterministic, contract-first database engine for mission-critical workloads. It keeps the durable scientific foundation of relational systems - relations, keys, constraints, algebraic reasoning, cost-based planning, transactions, recovery, and indexing - but rejects SQL ad hoc as the native application surface.

The native surface is **QUIC + custom RPC + Protobuf contracts + Procedure + SRPL**. QUIC provides the transport. The application protocol remains an Andromeda-specific protocol. Protobuf defines stable wire contracts and compatibility rules. gRPC is not part of the design. A client invokes only cataloged Procedures; each Procedure has a typed contract, a ContractHash, explicit result metadata, and a deterministic execution boundary.

SRPL, the **Strict Relational Procedure Language**, is the procedural relational language of the system. SRPL is not SQL renamed. It is a strict, typed, bounded, transaction-first language designed to compile to a semantic relational IR and then to procedure plans. The language should be readable, but it must never become ambiguous pseudo-natural language. The system must use SRPL to reduce ambiguity before execution.

The architecture is split into functional engines and transverse planes. Functional engines own clear responsibilities. Transverse planes impose cross-cutting rules such as durability, security, resource governance, policy, observability, and contract integrity. This avoids a God Engine and makes recovery, testing, and audit tractable.

The first implementation target is not GPU analytics, distributed multi-primary, or learned optimization. The first target is a recoverable vertical slice: Catalog, minimal SRPL, Transaction Kernel, WAL, Recovery, and contractual RPC. Everything else becomes credible only after this slice can create objects, execute a Procedure, commit durably, emit a ResultStream, and recover after a crash.

2. Document scope and evidence model

Purpose

This document consolidates the source corpus, project conversations available in context, and external research references into one master architecture reference. It is not a final product specification. It is a design baseline that makes the system boundaries, invariants, risks, and required specifications explicit.

Normative language

The words **must**, **must not**, **should**, **should not**, and **may** are used with their ordinary engineering meaning in this draft. A later standards-track specification should formalize them using an RFC-style convention.

Evidence model

Project sources are cited with source tags such as `SRC-01`. External references are cited with tags such as `EXT-QUIC-9000`. Design decisions use `DEC-xxx`. Risks use `RISK-xxx`. This keeps the PDF readable while preserving traceability.

Integration limits

The Markdown files and PDFs present in the workspace are directly integrated. Chat-derived content is integrated from the available project context and memory, not from a full raw export of every prior transcript. This limitation is recorded so that later iterations can attach verbatim chat ledgers if required.

3. Consolidated source ledger

Tag	Source	How this document uses it
SRC-00	00.Index.md	General V0 index, project intent, component map, and reading rule.
SRC-01	01.Vision_Doctrine_Invariants.md	Vision, doctrine, non-negotiable invariants, acceptance criteria, and criticality scale.
SRC-02	02.Lexique_Andromeda_SQL_Equivalences.md	Native vocabulary and controlled SQL equivalences.
SRC-03	03.Architecture_Globale_Modules.md	Global architecture, functional engines, transverse planes, and dependency prohibitions.
SRC-04	04.Catalogue_Bases_Namespaces_Objets.md	Canonical hierarchy: instance, system database, user databases, namespaces, tables, maps, enums, structured objects, and procedures.
SRC-05	05.SRPL_Langage_Procedures.md	SRPL role, procedure syntax, implicit transaction model, errors, time, mutations, result streams, and compilation path.
SRC-06	06.Type_System_Tables_Enums_StructuredObjects.md	Type system, numeric widths, decimal/float policy, bool, text, nullability, enum flags, and structured objects.
SRC-07	07.Maps_Analytique_Temps_Reel.md	Maps as materialized projections, coherence modes, analytics, and refresh policies.
SRC-08	08.Transaction_Kernel_WAL_MVCC_Recovery.md	Transaction kernel, WAL invariant, MVCC, write pipeline, recovery, and critical cases.
SRC-09	09.Storage_Engine_HotCold_BinaryFormats.md	Hot/cold storage, page/extents, manifest, snapshot publication, and I/O priority model.
SRC-10	10.Network_QUIC_RPC_Contracts.md	QUIC surfaces, custom RPC framing, metadata-before-payload, structured object transport, and backpressure.
SRC-11	11.Procedure_Store_Optimizer_Stats_Benchmark.md	Procedure Store, optimizer, statistics, predictive evidence, plan classes, and guardrails.
SRC-12	12.Security_IAM_Admin_Audit.md	Security doctrine, certificate identity, UserPrincipal, permissions, administration surface, debug isolation, and audit.
SRC-13	13.HADR_Backup_Restore_Forensic.md	Single primary plus replicas, quorum, failover, WAL shipping, backup, PITR, and forensic startup.
SRC-14	14.Hardware_Rust_CPU_GPU_NVMe.md	Rust, unsafe discipline, CPU profiles, GPU constraints, NVMe/SSD, ZNS, and HDD policy.
SRC-15	15.Modelization_Import_Batch_Catalog_Evolution.md	Modelization, DefinitionBatch, ordered import, DryRun, conflicts, transactional apply.
SRC-16	16.Roadmap_V0_Risques_Decisions.md	Roadmap, phases, risks, and open decisions.
SRC-17	17.Sources_References.md	Serious references: Codd, ARIES, Volcano, column stores, SQL Server, QUIC, Rust, NVMe/ZNS, CPU/GPU.
SRC-18	Andromeda_Storage_Engine.md	File-system, segments, contiguity invariants, append-only write workflow, and shutdown rules.
SRC-19	Rapport consolidé...pdf	Research synthesis on RDBMS, ACID, transactions, relational algebra, normalization, statistics, and indexing.

Tag	Source	How this document uses it
SRC-20	Corpus de référence...pdf	Reference corpus that separates stable foundations from modern 2024-2026 research fronts.
SRC-21	Fondation de SRPL...pdf	SRPL foundation, SQL critique, strict language principles, typing, syntax, compilation, plan cache, and governance.
CHAT-CTX	Project conversation context	Consolidated project memory: custom QUIC contracts, typed RPC, C# source generator relevance, storage hot/cold thinking, benchmark evidence, and SRPL constraints.

4. Design doctrine

Positioning

Andromeda is relational in its foundation, transactional in its guarantees, typed in its contracts, deterministic in its critical paths, compilable in its procedures, observable in its effects, and strict at its external boundaries. It is not a general SQL engine, an ORM, an object database, an application runtime, or an autonomous AI system.

Central doctrine

The operating doctrine is: **strict at the boundaries, adaptive inside the boundaries**. Boundaries include contracts, types, procedures, transactions, durable storage, security, network protocol, catalog, imports, permissions, and audit. Adaptation may occur in plan choice, batch size, row/column layout, caching, specialization, GPU use, statistics refresh, predictive benchmark, and resource policy. Any adaptation must be versioned, observable, bounded, and removable without breaking correctness.

Non-negotiable invariants

Invariant	Meaning
No SQL ad hoc application surface	Application execution goes through cataloged Procedures only.
Procedure-only behavior	The Procedure is the only callable behavioral unit.
Typed and hashed contract	Every Procedure contract has an exact canonical representation and ContractHash.
Implicit transaction	Every Procedure executes inside an engine-owned transaction scope.
Visible commit equals durable WAL	No mutation becomes visible until the durable log covers it.
RAM is never truth	RAM can cache or stage state, but it is not the reconstructible source of truth.
Cold snapshot plus WAL is truth	The recoverable state is the last valid cold snapshot plus durable WAL after it.
GPU outside commit path	GPU may accelerate analytics and evidence, not commit, rollback, WAL, or recovery.
Predictive evidence does not decide	Evidence informs the optimizer; the optimizer decides under policy.
Observable critical decisions	Any critical decision must be explainable after the fact.

Acceptance gate

A feature is rejected unless it can be defined, typed, bounded, observed, versioned, secured, recovered after crash, explained, and disabled without corrupting the engine. This gate is stricter than typical product feature review because the target is mission-critical infrastructure.

5. System overview

The system is organized around a small number of runtime truths: the catalog defines, security authorizes, execution orchestrates, the transaction kernel guarantees, storage persists, statistics inform, the optimizer decides, the Procedure Store remembers, predictive evidence proposes, and observability proves.

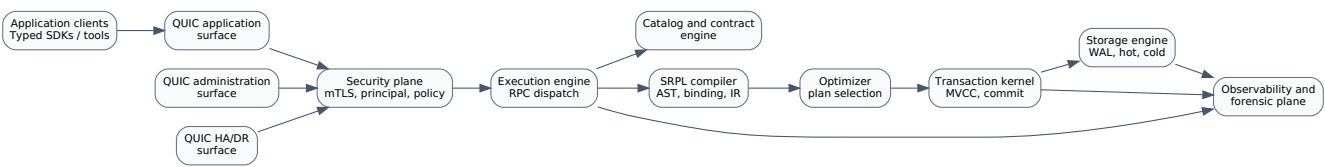


Figure 1. Consolidated Andromeda system overview.

Major surfaces

Surface	Purpose	Allowed capability	Explicitly excluded capability
Application surface	Execute application Procedures and read visible contracts.	HELLO, AUTH, GET CONTRACTS, EXECUTE PROCEDURE, STREAM RESULT, ERROR, minimal telemetry.	Object creation, administration, cluster operations, maintenance, deep debug.
Administration surface	Operate, inspect, maintain, and secure the instance.	DefinitionBatch import, backup/restore, plan inspection, SRPL debug on isolated snapshots, certificate and permission management.	Application traffic mixing and silent audit bypass.
HA/DR surface	Replicate and coordinate nodes.	WAL shipping, snapshot shipping, quorum, fencing, promotion, health state.	User Procedure execution and arbitrary administration.

Runtime planes

Transverse planes are not parent engines. They impose rules across engines. The important planes are durability, security, resource governance, policy, observability, and contract integrity. A design that centralizes these as one meta-engine becomes harder to test and easier to bypass.

6. Functional engine architecture

Engine map

Engine	Primary responsibility	V0 completion signal
Core Engine	I/O abstractions, clocks, hardware profile, memory accounting, scheduler integration.	Hardware profile is detected and exposed to policy without assuming a CPU or storage capability.
Network Surface Layer	Owns QUIC surfaces, sessions, streams, framing, quotas, and backpressure.	A Procedure can be executed over local QUIC framing with contract verification.
Catalog and Contract Engine	Owns canonical object definitions, types, Procedure contracts, versions, and ContractHash values.	CatalogVersion changes are transactional, WAL-covered, and auditable.
SRPL Compiler	Parses, binds, validates, and lowers SRPL to relational IR.	A minimal Procedure compiles into a typed semantic model and plan candidate.
Execution Engine	Dispatches RPC, validates contract, binds Procedure, orchestrates plan execution.	An invocation produces ResultStream metadata and completion state.
Transaction Kernel	Owns transaction states, visibility, commit/rollback, and concurrency semantics.	Commit visible = durable WAL is enforced.
Storage Engine	Owns WAL, pages, extents, BufferPool, HotStore, ColdStore, manifests, recovery.	Crash/recovery test passes for a committed Procedure.
Optimizer	Chooses Procedure plans under catalog, statistics, evidence, policy, and stability constraints.	A bounded plan cache exists and plan decisions are traceable.
Statistics Engine	Builds histograms, cardinality, skew, density, storage temperature, and StatsVersion.	A published StatsVersion can feed plan choice.
Predictive Evidence Engine	Runs controlled scenarios and produces evidence, not decisions.	ScenarioEvidence is generated, scored, expired, and ignored if invalid.
Security/IAM Plane	Binds certificates to principals and evaluates roles, permissions, and policies.	No execution occurs without a principal and permission decision.
Observability/Forensic Plane	Correlates traces, metrics, decisions, recovery evidence, and audits.	Each critical path emits traceable evidence.

Dependency prohibitions

The design must prohibit dependencies that widen attack surface or compromise recovery. The Application surface must not call Administration or Cluster capabilities. GPU runtime must not enter commit protocol. Predictive evidence must not force a plan. A Procedure must not call external network or filesystem resources. A certificate must not bypass permission checks. A replica must not self-promote without quorum.

7. Rust implementation baseline

Why Rust

Rust is the primary implementation language because it fits a mission-critical concurrent kernel better than languages that depend on garbage collection or unchecked aliasing by default. It gives ownership, borrowing, deterministic drop semantics, and a strong type system while still allowing low-level control where justified.

Edition and runtime

The recommended baseline is Rust 2024 for new crates, with a runtime boundary designed explicitly. Tokio is a strong default candidate for asynchronous network and I/O scheduling, but the database kernel must avoid leaking arbitrary async complexity into transaction and storage invariants. Execution boundaries should be explicit, cancellable, observable, and resource-accounted.

Unsafe Rust policy

Unsafe Rust is expected in binary layouts, mapped pages, direct I/O, FFI, GPU integration, CPU intrinsics, and specialized allocators. It must be isolated behind safe APIs, documented with invariants, fuzzed, property-tested, code-owned, and audited. Panic in critical server paths should be treated as a defect unless the path is explicitly process-fatal and recoverable at startup.

Crate workspace strategy

Crate family	Purpose	Rule
andromeda-core	clocks, errors, identifiers, resource accounting, common policy	No storage or network dependencies.
andromeda-proto	generated Protobuf code and wire compatibility helpers	No domain logic; conversion to internal types is explicit.
andromeda-quic	QUIC session, stream, frame, and backpressure logic	No catalog mutation logic.
andromeda-catalog	object model, CatalogVersion, ContractHash, dependency graph	No direct network transport logic.
andromeda-srpl	lexer, parser, AST, binder, semantic IR	No physical storage code.
andromeda-tx	transaction state machine, MVCC rules, commit protocol	No GPU dependency.
andromeda-storage	WAL, pages, extents, manifests, recovery	No application Procedure semantics.
andromeda-exec	invocation orchestration and operator runtime	Depends on catalog, SRPL, optimizer, transaction, storage.
andromeda-observe	tracing, metrics, audit event schemas	Stable event model; no critical decisions.

CPU and hardware profiles

The engine must detect x64 and ARM64 capabilities at runtime and apply policy. It must not assume AVX2, AVX-512, AMX, NEON, SVE, or SVE2. Capabilities must be reflected in HardwareProfile and used by operators only through gated dispatch.

8. QUIC plus Protobuf without gRPC

Decision

The protocol stack is now: **QUIC transport + Andromeda custom application protocol + Protobuf contracts**. This is not gRPC. gRPC is rejected because it would impose a framework model that is not aligned with the database-specific requirements: exact metadata before payload, explicit RowCountExact when required by contract, multi-result streaming, typed structured objects, plan and catalog version binding, and narrow Procedure-only behavior.

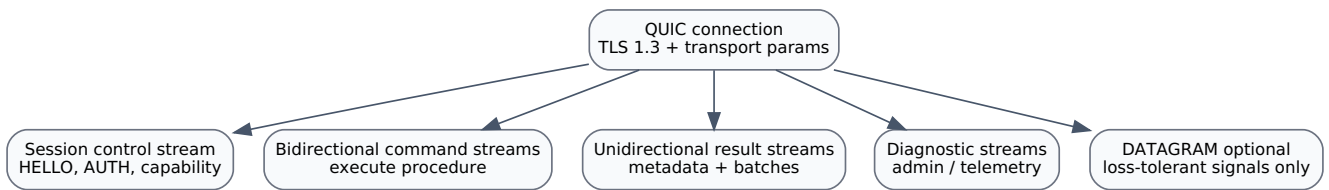


Figure 2. QUIC stream taxonomy for Andromeda.

Responsibilities by layer

Layer	Owns	Must not own
QUIC	Connection establishment, streams, flow control, congestion behavior, encryption integration, connection IDs, migration rules.	Database contracts, catalog semantics, Procedure authorization.
Andromeda frame	Frame type, RequestId, SessionId, TxId, PayloadLength, flags, CRC/header validation, routing, batch boundaries.	Schema evolution policy by itself.
Protobuf contract	Stable message schemas, field numbers, presence, compatibility, command metadata, contract manifests, error envelopes, typed payload descriptors.	Internal domain invariants or physical page layout.
Internal Rust model	Catalog objects, Procedure contracts, execution IR, transaction state, storage records.	Wire compatibility requirements leaking into every internal structure.

Stream policy

Correctness-critical data must use reliable QUIC streams. QUIC DATAGRAM is allowed only for loss-tolerant soft-state signals, never for Procedure results, transaction decisions, WAL shipping, catalog mutation, or security decisions. 0-RTT must remain disabled by default for any request that could mutate state, consume nonce-like capability, or produce non-idempotent behavior.

Protobuf contract policy

Use Protobuf for interchange contracts, not as the internal domain model. Reserve deleted field numbers and names. Avoid reusing tags. Treat field presence deliberately. Keep packages stable. Prefer additive evolution. Version the canonical descriptor set and bind it to ContractHash and ProtocolLayout. JSON mapping is allowed for diagnostics only, not as the normative wire path.

Frame and message relation

```
QUIC stream bytes
-> Andromeda FrameHeader
    { frame_type, request_id, session_id, tx_id, payload_length, flags, header_crc }
-> Protobuf message envelope
    { protocol_version, contract_hash, catalog_version, payload_kind, payload_descriptor }
-> Protobuf payload or binary typed batch
    { parameters, structured object batch, result metadata, result batch, completion }
```

Metadata before payload

Result metadata must precede result batches. For contractual result sets, RowCountExact or TotalRowsExact must be known and sent before payload streaming. This requirement is intentionally opinionated: it enables client preallocation, deterministic result validation, and typed SDK generation, at the cost of extra work for some plans.

9. Procedure contract design

Contract identity

A Procedure contract is not only a name and parameter list. It is a canonical, versioned description of callable behavior. It includes input parameters, structured object contracts, output shapes, result streams, result metadata policy, required permissions, transaction policy, isolation policy, protocol layout, compatibility policy, and hashes.

Field	Purpose
ProcedureName	Human and catalog identity.
ProcedureId	Stable internal identifier.
ContractHash	Canonical hash over the stable contract representation.
CatalogVersion	Catalog version used for binding.
StatsVersion	Statistics version that may affect planning.
InputShape	Scalar parameters and their types.
StructuredObjectShape	Tabular parameter shape, uniqueness, ordering, row count policy, and layout policy.
OutputShape	Result stream count, column descriptors, order contract, and error model.
RequiredPermissions	Permissions required before invocation.
TransactionPolicy	Implicit transaction properties, access mode, isolation, retry rules.
ProtocolLayout	Frame and Protobuf message layout used on the wire.
CompatibilityPolicy	Backward/forward compatibility expectations.

SDK and source generator use

The contract endpoint is well suited to typed client generation. In a C# backend, a source generator can consume Procedure manifests, generate request/response records, create serializers, validate ContractHash at startup, and fail fast if the application was built against a stale catalog contract. The database remains the contract authority.

Multi-result policy

A Procedure may return multiple result streams, but each stream must have explicit metadata and completion. The client must be able to validate the expected stream sequence and reject unexpected shapes.

10. SRPL language model

Language role

SRPL means **Strict Relational Procedure Language**. It is a language for transaction-bound relational Procedures. It is declarative where possible, procedural only where necessary, typed, bounded, and compiled through semantic stages. It is not a SQL dialect.

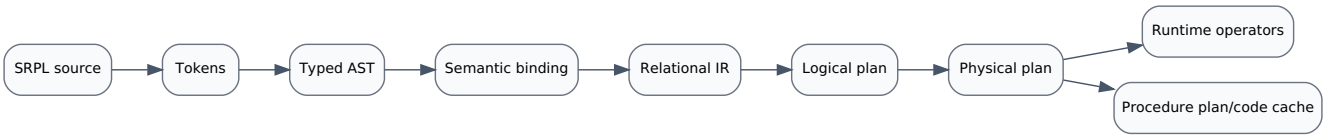


Figure 3. SRPL compilation and execution pipeline.

Core principles

Principle	Consequence
Procedure-first	A user calls a Procedure, not an ad hoc query.
Set semantics by default	Bag semantics must never be implicit.
Explicit absence	Nullable ambiguity is replaced by rare, explicit, typed absence.
Bool is required	No three-valued boolean logic in the core language.
Cardinality is contract data	one, optional one, many, and nonempty many should become compiler-visible concepts.
Lexical scope is strict	A name declared in a block lives in that block only.
Bounded loops only	General unbounded loops, free recursion, filesystem access, external network access, and non-deterministic random are excluded.
Engine-owned time	current transaction timestamp, current invocation timestamp, and monotonic epoch are produced and traced by the engine.
Semantic IR first	Optimization targets the IR, not the surface text.

Canonical Procedure sketch

```
procedure Inventory.ReserveStock
accepts
  ProductId i64,
  Quantity i32
returns
  Reservation collection of ReservationResult
begin
  ensure Quantity > 0
    else fail InvalidQuantity "Quantity must be greater than zero";

  ensure exists Product where Id = ProductId
    else fail ProductNotFound "Product does not exist";

  update Stock
```



```
    set AvailableQuantity = AvailableQuantity - Quantity
  where ProductId = ProductId
    and AvailableQuantity >= Quantity;

  ensure affected rows equals 1
    else fail InsufficientStock "Not enough stock";

  return Reservation from
    select ProductId,
      Quantity,
      current transaction timestamp as ReservedAt;
end;
```

Recommended evolution

The current V0 syntax can evolve toward a more contract-shaped American English style without losing strictness. The key is to keep the semantic core stable: typed parameters, explicit reads/writes, explicit cardinality, explicit absence, explicit result shapes, and a canonical IR.

11. Type system and data model

Purpose

The type system reduces ambiguity before execution. Every critical value must be typed, bounded, serializable, comparable under explicit policy, and verifiable.

Scalar types

Family	V0 direction	Critical rule
Signed integers	i8, i16, i32, i64, optional i128 if justified.	Width is explicit; dangerous implicit conversion is rejected.
Unsigned integers	u8, u16, u32, u64, optional u128 if justified.	Use only when the domain is truly non-negative.
Decimal	decimal.min, decimal.mid, decimal.max, decimal.custom(precision, scale).	Use for money, accounting, exact quantities, and invariants.
Float	float.min, float.mid, float.max, float.custom(bits, mode).	Use for measurements, scores, statistics, and analytics; never for keys, money, or exact invariants.
Bool	bool required.	Nullable bool is forbidden.
Text	utf8, utf16, unicode.	Encoding, length policy, collation/comparison, and normalization must be explicit where relevant.
Timestamp	Engine-provided transaction timestamp, invocation timestamp, monotonic epoch.	Raw ambient now() semantics are unsafe in deterministic procedures.

Nullability

Nullability is rare and explicit. It represents a true business absence, not a shortcut for incomplete modeling. The language should prefer explicit optional types or absence states over ambient NULL-like behavior.

StructuredObject

A StructuredObject is a typed tabular shape used as an input, output, or structured parameter. It may be generic or relational. The core rule is: StructuredObject owns intrinsic shape invariants; Procedure owns contextual invariants.

```
structured OrderLineInput
{
  ProductId i64 references Inventory.Product.Id;
  Quantity i32;
}
unique by ProductId;
```

Protocol cardinality

A StructuredObject carried over the protocol must include name, ContractHash, RowCountExact, column descriptors, layout, payload length, and batch descriptors. Layout can be row-major, column-major, or hybrid. The engine should classify shapes as small, medium, large, skewed, unique, duplicated, and sorted to guide plan selection.

12. Catalog and metadata architecture

Canonical hierarchy

The System Database is the internal master. It contains database registry, principal registry, certificate registry, global type registry, engine policy registry, hardware profiles, security audit, catalog history, HA/DR state, and instance configuration. Any mutation of the System Database is C5-critical and must be audited and WAL-covered.

User database object model

Object	Storage	Contract	Statistics	Callable
Table	Yes	Yes	Yes	No
Map	Yes	Yes	Yes	No
Enum	Type-cataloged	Yes	Optional	No
StructuredObject	Non-persistent	Yes	Possible scenarios	Parameter / result shape
Procedure	Code / plan / contract	Yes	Via Procedure Store	Yes

Versioning

Every catalog mutation produces a new CatalogVersion. The engine must be able to reconstruct which object existed with which contract, at which version, introduced by which DefinitionBatch, and authorized by which principal.



Figure 4. Modelization and DefinitionBatch import pipeline.

Modelization

Modelization is an external design artifact, not a runtime database object. It becomes real only through an ordered, validated, auditable, transactional DefinitionBatch. The term Model is rejected as a native term because it collides with DTO, class, and record concepts.

13. Transaction kernel

Authority boundary

The Transaction Kernel owns transaction state. The Execution Engine orchestrates, and the Storage Engine persists and reconstructs. This boundary is essential: orchestration must not redefine commit, and storage must not invent visibility.

State machine

```
Created -> Active -> Committing -> Committed -> Disposed
Created -> Active -> Failed -> RollingBack -> RolledBack -> Disposed
Active -> Poisoned -> RollingBack -> RolledBack -> Disposed
```

Commit invariant

Visible commit = durable WAL. No mutation becomes visible until the durable LogStream records the required transaction state. This invariant is more important than any single storage layout choice.

Write pipeline

```
Procedure Invocation
-> TransactionScope
-> RAM modifications
-> WAL records
-> durable WAL flush
-> visible commit
-> dirty page writeback to HotStore
-> logical checkpoint
-> cold snapshot publication
```

MVCC visibility

```
RowHeader {
    RowId,
    BeginTs,
    EndTs,
    CreatorTxId,
    DeleterTxId,
    PrevVersionPtr,
    Flags
}

visible(row, snapshotTs) =
    row.BeginTs <= snapshotTs
    and (row.EndTs == INF or row.EndTs > snapshotTs)
```

Rollback and poison

A poison state marks an invocation or transaction path as unsafe to continue normally. Poison must be observable, auditable, and always followed by a controlled rollback or fail-stop behavior.

14. Storage architecture

Hot and cold truth model

The storage model is **RAM active -> NVMe transactional/hot -> HDD cold consolidated**. HDD must not receive direct OLTP pressure. RAM is volatile. NVMe carries WAL, hot pages, version store, temp store, statistics builds, benchmark temp, and optional Procedure code/cache. HDD carries published snapshots, manifests, archives, cold indexes, and published statistics.

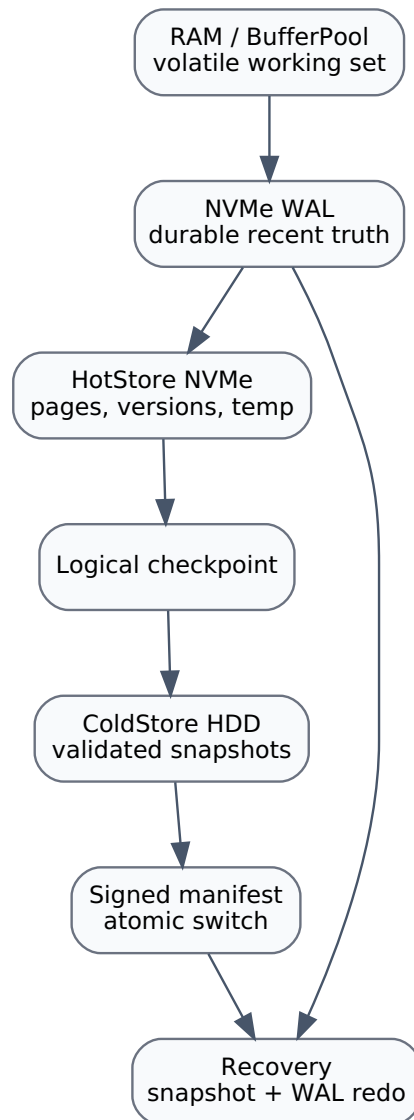


Figure 5. Hot/cold storage and recovery model.

Page and extent policy

Component	V0 direction
Page size	16 KiB or 32 KiB should be evaluated; the final choice remains open.
Extent	64 pages is the current recommended conceptual extent.
Page header	

Component	V0 direction
	Magic, FormatVersion, PageType, FileId, PageId, ObjectId, AllocationUnitId, PageLsn, PageEpoch, Prev/Next page IDs, offsets, free bytes, slot count, row count, flags, header CRC.
Page trailer	PayloadCrc64, PageHash, torn-write guards.
Fixed row page	OccupancyBitmap + dense fixed rows.
Hybrid row page	FixedPartArea + VariableHeap + OffsetDirectory + OverflowPointers.

ColdStore contiguity

ColdStore is immutable after publication. A cold page is never physically split after it is written. A modification creates a new version and writes a new segment or append-only zone. No update-in-place is allowed in ColdStore.

Segment model

```
Manifest
  -> SegmentIndex
    -> SegmentFiles
      -> Blocks
        -> Pages

Segment = [Header][Block][Block][Block][Trailer]
Block   = BlockHeader + Payload + BlockTrailer
Page    = PageHeader + RowData + SlotDirectory + PageTrailer
```

Snapshot publication

Snapshot publication must use staging, validation, manifest.next, fsync, and atomic switch. At all times, at least one valid cold snapshot must remain available. Cleanup is delayed until the new manifest and its referenced files are fully validated.

15. Recovery and forensic startup

Recovery truth

The canonical reconstructible state is the last valid cold snapshot plus durable WAL since that snapshot. If a manifest is corrupt, the previous manifest is attempted. If WAL is truncated, recovery reads until the last CRC-valid record and reports the boundary. If NVMe is lost, the engine returns to the last HDD snapshot unless a mirrored WAL path exists.

Recovery sequence

1. Read manifest.
2. Verify signature and hashes.
3. Mount cold snapshot.
4. Read WAL from RequiredWalStartLsn.
5. REDO committed durable records.
6. Undo or discard incomplete transactions.
7. Rebuild hot maps and volatile structures.
8. Verify catalog and storage invariants.
9. Open the database in the selected startup mode.

Startup modes

```
{table(['Mode','Behavior'], [ ('FastStart','Open with standard recovery when invariants are clean and no forensic condition is detected.'), ('SafeStart','Open conservatively with additional checks and restricted behavior.'), ('ForensicStart','Block application connections, perform consistency analysis, and emit a forensic report.'), ], 'dense')}
```

16. Maps and internal analytics

Definition

A Map is a materialized projection: relational definition + storage + statistics + contract + coherence policy. There is no generic virtual View in the native model.

Map classes

Map class	Purpose	Typical refresh
Projection Map	Materialized projection over one relation.	Immediate or Incremental.
Join Map	Materialized join shape for critical procedures.	Incremental or Deferred depending on cost.
Aggregate Map	Precomputed aggregates.	Incremental recommended.
Analytical Map	Large analytical projection or columnar structure.	Deferred or SnapshotOnly.
Security Map	Materialized security projection.	Immediate or policy-defined.

Coherence modes

Immediate means the Map participates in the same TransactionScope as the base mutation. Incremental uses a controlled delta log and apply phase. Deferred postpones maintenance under a policy and budget. SnapshotOnly is rebuilt for analytical snapshots.

GPU boundary

Maps may carry histograms, cardinality, skew, compression, access paths, column segments, and GPU-generated statistics. GPU work must stay outside commit path and must be validated before publishing a StatsVersion or Map state that affects planning.

17. Procedure Store, optimizer, statistics, and evidence

Procedure Store

Procedure Store is the real memory of Procedure invocations. It replaces the Query Store vocabulary while preserving the useful concept of historical plan and runtime evidence.

Stored field group	Examples
Identity	InvocationId, ProcedureId, ContractHash, CatalogVersion, StatsVersion, PlanId, PlanClass.
Input shape	InputShapeHash, StructuredObjectShapeHash, parameter classes, row counts.
Work done	RowsRead, RowsReturned, RowsWritten, RowsAffected, LogicalReads, PhysicalReads.
Resource use	Duration, CacheHits, WalBytes, TempBytes, SpillBytes.
Outcome	ErrorKind, completion status, transaction state, rollback reason if any.

Optimizer model

The optimizer is cost-based, evidence-based, workload-aware, stability-aware, and bounded. It consumes Catalog, constraints, StatsVersion, Procedure Store, ScenarioEvidence, HardwareProfile, StorageTemperature, and PolicyVersion.

Plan classes

Andromeda should support bounded multi-plan behavior: Generic, Small, Medium, Large, Skewed, StructuredObjectSmall, StructuredObjectLarge, and Maintenance. Quotas, dominance pruning, eviction, and hysteresis are mandatory.

Predictive evidence

Predictive Evidence explores what could happen. It does not decide. A ScenarioEvidence record contains scenario identity, Procedure and object identity, ContractHash, CatalogVersion, StatsVersion, workload intent, parameter shape, estimated cardinality, estimated logical cost, I/O tier impact, spill risk, and separate confidence, criticality, freshness, realism, and stability scores.

Statistics publishing

Statistics candidates must be collected, computed, validated, and published through a controlled switch. GPU may accelerate computations, but CPU validation and deterministic publishing remain required.

18. Security, IAM, administration, and audit

Security chain

The security model is certificate identity to logical principal to roles/groups to permissions to policies to audit. A certificate is not the user. This separation allows rotation, revocation, expiration, multi-device mapping, human/service separation, and long-term audit.



Figure 6. Security decision chain.

Surface scope

A CertificateIdentity includes a SurfaceScope: Application, Administration, Cluster, BackupAgent, or MonitoringAgent. SurfaceScope constrains which QUIC surface and command family the certificate can use before logical permissions are even evaluated.

Permissions

Permission family	Examples
Application	ExecuteProcedure, ReadContract.
Definition	CreateTable, CreateMap, CreateProcedure, ImportDefinitionBatch.
Diagnostics	DebugProcedure, ReadProcedureStore, inspect plans.
Security	ManageSecurity, rotate certificates, revoke CertificateIdentity.
Recovery	Backup, Restore, ForensicStart.
Cluster	ClusterPromote, fence node, update cluster manifest.

Audit events

SecurityAuditTrace, CatalogChangeTrace, AdminOperationTrace, ProcedureInvocationTrace, TransactionTrace, PlanDecisionTrace, RecoveryTrace, ClusterEventTrace, and DefinitionBatchApplyTrace are mandatory event families. No global audit disable switch is allowed.

Debug isolation

SRPL debug runs on an isolated recent snapshot. It must never mutate active production state. Debug is an Administration surface capability, not an Application surface feature.

19. HA/DR, backup, restore, and forensic operations

V0 topology

V0 uses **Single Primary + Replicas**. Multi-primary is explicitly out of scope for V0. Roles include Primary, ReplicaPreferred1, ReplicaPreferred2, ReplicaN, and optional Witness.

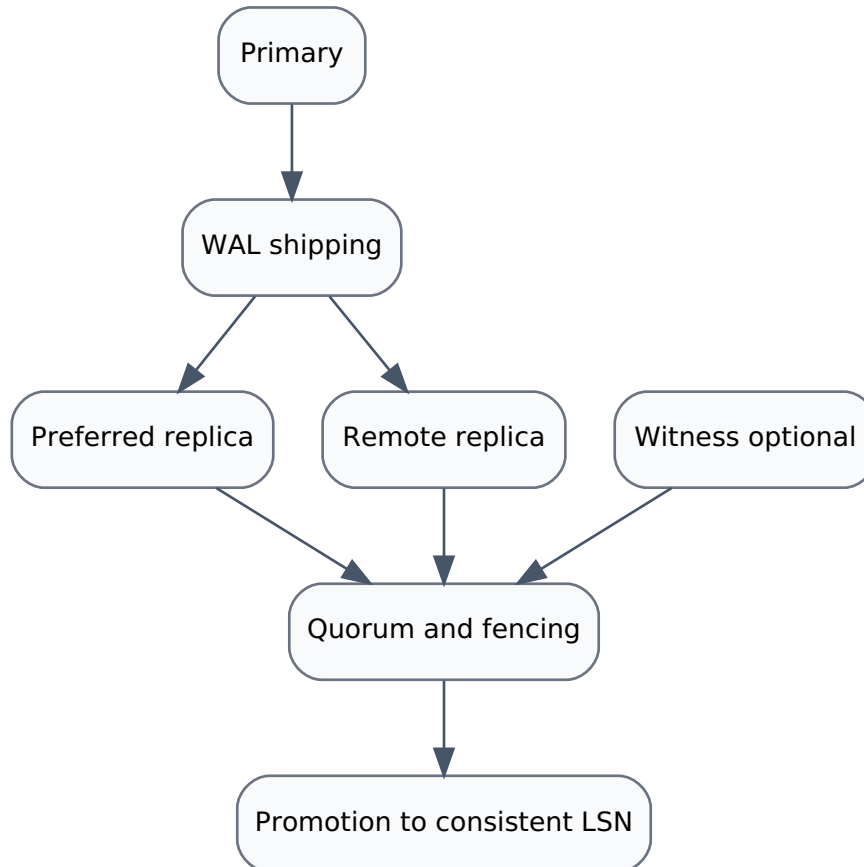


Figure 7. HA/DR V0 topology and promotion flow.

Quorum and fencing

Quorum verifies majority or configured rule, suspected primary fencing, most advanced safe LSN, promotion eligibility, node priority, divergence, and capacity to accept the role. A replica must never self-promote without quorum.

Replication

WAL shipping is the primary V0 mechanism. Snapshot shipping is used for bootstrap and resynchronization. Synchronous replication may be used for a preferred replica. Asynchronous replication may be used for remote replicas. RPO and RTO must be explicit contract parameters, not marketing labels.

Backup is not HA/DR

HA/DR protects availability. Backup protects against human error, logical corruption, deletion, ransomware, application bugs, bad migrations, and policy failures. Both are required and must be tested separately.

PITR

Point-in-time restore selects a cold snapshot, chooses a target LSN, replays WAL, cancels incomplete transactions, validates the manifest, and opens in test or production mode.

20. Observability and operability

Native observability

Observability is a correctness feature. A mission-critical database must prove what happened, what plan was selected, which policy allowed it, which transaction committed, which WAL range covers it, and how recovery rebuilt it.

Trace hierarchy

```
SessionTrace
-> ProcedureInvocationTrace
  -> ContractValidationTrace
  -> PlanDecisionTrace
  -> TransactionTrace
    -> WalTrace
    -> MvccTrace
  -> ResultStreamTrace
-> SecurityAuditTrace
-> ResourceTrace
```

Metrics

Metric group	Examples
Network	active sessions, streams, backpressure events, frame rejects, QUIC connection close reasons.
Execution	Procedure duration, rows read/returned/written, plan class, operator latencies.
Storage	WAL bytes, WAL flush latency, hot page reads/writes, cold snapshot duration, checkpoint time.
NVMe/SSD	bytes written, write amplification, wear percentage, spare available, unsafe shutdowns, media errors, temperature.
Optimizer	cardinality error, plan invalidations, plan reuse, evidence confidence, spill risk.
Security	auth failures, denied permissions, certificate expiration, revocations, break-glass events.

Explainability

DecisionTrace is the union of explanation, telemetry, and audit. It should not be a cosmetic EXPLAIN output. It must be sufficient to reconstruct why the optimizer chose a plan and why security allowed or denied a call.

21. Hardware acceleration boundaries

CPU

The CPU remains the critical path authority. WAL checksum, encryption, short scans, bitmap/cardinality operations, compression, and latch/counter operations may use CPU-specific acceleration only after runtime capability detection and policy approval. Crypto must use vetted libraries and hardware acceleration where available; never implement custom cryptography.

GPU

Allowed GPU use	Forbidden GPU use
Histograms, cardinality, distribution, skew detection.	Commit protocol.
Analytical scans and large aggregations.	WAL generation or durable flush.
Predictive benchmark scenarios.	Rollback and recovery.
Column segment analysis.	Row-by-row OLTP index lookup.
Large batch statistics builds.	Short MVCC visibility checks.

NVMe, SSD, and ZNS

NVMe and SSD support hot transactional state: WAL, hot pages, version store, temp store, stats build, benchmark temp, and optional Procedure code cache. Zoned Namespaces are attractive for segmented append-only WAL, snapshot build, cold construction, and controlled compaction, but ZNS should be optional in V0.

HDD

HDD is ColdStore. It carries snapshots, archives, cold indexes, manifests, and published statistics. It must not carry active WAL as the only durable copy, random update critical paths, active version store, or temp-heavy workloads.

22. External research foundation

Stable foundation

The scientific foundation remains stable: Codd's relational model, relational algebra, cost-based optimization, transaction theory, WAL/ARIES recovery, normalization by dependencies, and mathematically grounded access paths such as B-tree and B+tree structures. Modern work extends this base; it does not erase it.

Research fronts

Area	Status for Andromeda
Learned cardinality estimation	Research-grade; useful as evidence after validation, not a replacement for robust histograms in the critical optimizer baseline.
Learned indexes	Research-grade; useful for bounded experiments, not a default replacement for B+tree-like ordered access paths.
Deterministic concurrency control	Relevant for selected distributed workloads, but not a universal replacement for MVCC/SSI/2PL-like mechanisms.
Hybrid transactional/analytical processing	Relevant for Maps and internal analytics, but OLTP recovery invariants remain primary.
Column stores	Relevant to analytical Maps and snapshots, not a reason to weaken row-level transactional correctness.
Latch-free or log-structured indexes	Relevant for hot/cold and modern hardware designs; must be evaluated with recovery and observability requirements.

Adoption rule

An external idea is integrated only if it can be defined, bounded, observed, versioned, tested, replayed, and disabled without breaking correctness. This rule prevents fashion-driven architecture.

23. V0 roadmap

Implementation order

Phase	Scope	Exit criteria
Phase 0 - Minimal specifications	Official lexicon, catalog object model, type system v0, Procedure contract v0, RPC frame v0, Protobuf schema v0, PageHeader v0, WalRecord v0, Manifest v0, transaction state machine.	Specification artifacts exist and have test vectors.
Phase 1 - Vertical prototype	Create database, namespace, table, enum, structured object, Procedure; call Procedure through local QUIC/RPC; durable WAL; commit; ResultStream; Procedure Store trace; crash/recovery test.	A crash after commit recovers visible state; a crash before commit does not publish partial state.
Phase 2 - Serious storage	BufferPool, HotStore abstraction, cold snapshot publication, atomic manifest switch, WAL replay, basic MVCC, logical checkpoint.	Recovery can mount snapshot + WAL and verify invariants.
Phase 3 - Minimal optimizer	Procedure Plan, Procedure Plan Cache, basic cost model, StatsVersion, Procedure Store feedback.	Plan decisions are traceable and invalidated correctly.
Phase 4 - Maps and advanced StructuredObject	Immediate simple Map, incremental Map, row/column StructuredObject layout, RowCountExact protocol.	StructuredObject batch path supports set-based execution and result metadata.
Phase 5 - Administration and security	mTLS, UserPrincipal, certificate registry, permissions, Admin surface, audit ledger, debug snapshot.	A denied command leaves trace; a debug command cannot mutate production.
Phase 6 - Analytics, benchmark, GPU	Optional GPU statistics, ScenarioEvidence, Predictive Evidence Engine, advanced Procedure Store.	Evidence informs optimizer without overriding policy.

Critical V0 tests

1. Create and mutate catalog objects through DefinitionBatch with rollback on failure.
2. Execute a minimal SRPL Procedure through QUIC framing with Protobuf envelopes.
3. Validate ContractHash mismatch and reject invocation before transaction creation.
4. Commit a mutation only after durable WAL flush.
5. Crash after WAL flush and before page writeback; recover committed state.
6. Crash before durable commit; recover without partial visibility.
7. Reject Application surface attempts to perform Administration commands.
8. Generate Procedure Store and DecisionTrace records for every invocation.

24. Open decisions and decision debt

Open decisions

Decision area	Current state	Recommended next action
Final product name	Andromeda is used as project name.	Keep until implementation repository exists; avoid renaming during specification writing.
SRPL concrete syntax	V0 syntax exists; stricter American-English contract syntax is proposed.	Freeze a grammar v0.1 after validating Procedure examples.
ContractHash format	Concept exists; binary canonicalization remains open.	Define descriptor-set canonicalization, byte order, hash algorithm, and collision policy.
Endianness	Open.	Use little-endian internally if chosen for hardware pragmatism, but document wire-level rules explicitly.
Page size	16 KiB or 32 KiB candidate.	Benchmark WAL/page/cache behavior before locking.
Decimal representation	Open.	Define precision/scale storage, overflow, rounding, and deterministic arithmetic rules.
Float modes	Open.	Separate approximate analytics from exact relational invariants.
QUIC implementation	Open.	Benchmark Quinn, s2n-quic, and MsQuic against latency, throughput, security, maintenance, and Rust integration.
Protobuf toolchain	Updated direction requires choice.	Evaluate official Rust Protobuf support versus prost capabilities, especially if Edition 2024 is required.
HA/DR quorum protocol	Single Primary + Replicas is chosen; detailed quorum is open.	Define fencing, LSN comparison, witness behavior, promotion rules, and split-brain tests.
Backup encryption and keys	Open.	Design key hierarchy and restore-time access policy before production backup design.
Procedure Code Cache	Optional.	Implement plan cache before native code cache; keep code cache behind policy.

Explicit challenge

The Protobuf update is architecturally sound only if Protobuf is kept as a wire contract format and does not become the internal domain model. If generated Protobuf structs leak into catalog, optimizer, storage, or transaction logic, the system will inherit wire evolution constraints inside its core. The correct pattern is conversion at the boundary.

25. Native vocabulary and naming discipline

Rule

The project uses a native vocabulary first and SQL equivalences second. The rule is: **one concept, one word, one meaning**. SQL terms are allowed only as explanatory bridges. They must not define the native surface.

Native term	Meaning	Pedagogical SQL equivalent	Discipline
Procedure	Only transactional behavioral unit callable by RPC.	Stored procedure.	Native term is Procedure, not query.
Invocation	One effective Procedure call.	Procedure execution.	Carries RequestId, SessionId, TxId, ContractHash, CatalogVersion.
Procedure Store	Historical memory of invocations and plan behavior.	Query Store.	Do not use Query Store as native wording.
Map	Materialized projection.	Materialized view.	There is no generic native View.
StructuredObject	Typed tabular parameter/result shape.	TVP.	Always typed and contract-bound.
Modelization	External design artifact before import.	Model or diagram.	Not a runtime object.
DefinitionBatch	Ordered catalog mutation batch.	Migration script / DDL batch.	Must be transactional, validated, and audited.
RowCountExact	Exact row count advertised before payload when contractual.	COUNT metadata.	May require extra work; intentionally deterministic.

Terms to avoid

Query, Query Store, View, Model, migration script, SQL ad hoc, dynamic SQL, and unbounded script are avoided in native specification text. They may appear in comparison sections only.

26. Wire protocol deep specification

Protocol objectives

The wire protocol must support persistent sessions, high-frequency RPC calls, large payloads, typed StructuredObjects, multi-result streams, backpressure, deterministic metadata, and strict contract validation. The design assumes professional deployments where client count is controlled, but payload volume and RPC frequency can be high.

Frame families

Frame family	Purpose	Reliability requirement
HELLO	Protocol version, capability discovery, max frame sizes, supported compression, supported Protobuf schema set.	Reliable stream.
AUTH	Authentication continuation after QUIC/TLS and certificate identity binding.	Reliable stream.
CONTRACT_REQUEST	Read Procedure manifests and contract descriptors.	Reliable stream.
CONTRACT_RESPONSE	Return manifest, ContractHash, ProtocolLayout, schema descriptors, and policies.	Reliable stream.
RPC_EXECUTE_REQUEST	Invoke a cataloged Procedure with scalar and structured parameters.	Reliable stream.
RPC_METADATA	Result stream metadata, column descriptors, row counts, and batch descriptors.	Reliable stream and before payload.
RPC_BATCH	A typed result or StructuredObject batch.	Reliable stream.
RPC_COMPLETION	Final status, RowsAffected, transaction outcome, and trace identifiers.	Reliable stream.
ERROR	Protocol, security, contract, execution, transaction, storage, or recovery error envelope.	Reliable stream.
TELEMETRY_SOFT_SIGNAL	Optional soft-state signal.	May use DATAGRAM only if loss is acceptable.

FrameHeader baseline

```

FrameHeader {
  FrameType    frame_type;
  u64          request_id;
  u64          session_id;
  optional u64 tx_id;
  u64          payload_length;
  u32          flags;
  u32          header_crc;
}

```

Backpressure model

Backpressure is not a network-only condition. It can be triggered by saturated receive buffers, a slow client, saturated execution queues, WAL flush lag, NVMe pressure, temp store quota, result spool growth, or catalog lock contention. The engine may slow reads, reduce batch size, spool under quota, reject new RPC calls, or close abusive sessions.

Cancellation model

Cancellation is a first-class protocol operation. It must identify RequestId, ProcedureId, and TxId if a transaction exists. Cancellation may stop result streaming, ask execution to stop, or trigger rollback depending on transaction state. The protocol must distinguish client cancellation, server cancellation, timeout, resource governance cancellation, and poison-triggered rollback.

27. Protobuf schema governance

Why Protobuf fits the updated direction

Protobuf fits the project when it is used as a stable binary contract language, not as an RPC framework and not as the internal database model. It gives field numbering, compact binary encoding, schema evolution patterns, generated code, and descriptor-based tooling. The project keeps QUIC and replaces neither QUIC nor the Andromeda frame model with gRPC.

Package organization

Package	Purpose
andromeda.protocol.v1	Stable wire envelopes, frame payload messages, error envelopes, session messages.
andromeda.contract.v1	Procedure manifests, contracts, input/output shapes, type descriptors, compatibility policy.
andromeda.catalog.v1	Exported catalog descriptors used by clients and tools, not the internal catalog representation.
andromeda.telemetry.v1	Public telemetry envelopes and trace identifiers.
andromeda.admin.v1	Administration surface messages; separated from application messages.
andromeda.cluster.v1	HA/DR and replication messages; separated from application and administration messages.

Schema evolution matrix

Change	Compatibility	Rule
Add optional field	Usually backward/forward compatible.	Allowed with a new field number.
Rename field	Wire-compatible but source-breaking.	Allowed only if descriptors preserve semantic name mapping or a new field is used.
Remove field	Dangerous.	Reserve field number and name; keep reader behavior documented.
Change field type	Generally breaking.	Use a new field number.
Reuse field number	Forbidden.	Never reuse tags.
Move to oneof	Potentially breaking.	Treat as new schema unless proven compatible.
Change enum numbering	Breaking.	Never change numeric values; reserve removed values.

Descriptor set as contract artifact

The build process must emit a descriptor set for wire contracts. The descriptor set hash participates in ProtocolLayout and may feed ContractHash. Client SDKs must validate expected hashes at startup or deployment time.

28. Typed client and C# source generator strategy

Client generation target

The database exposes canonical contracts. Backends consume them. A C# source generator can turn Procedure manifests into strongly typed client wrappers, request records, result records, batch readers, and contract validators.

Generated artifacts

Artifact	Purpose
Procedure client method	Strongly typed method per Procedure with cancellation and timeout support.
Input record	Immutable scalar parameter record with validation attributes derived from contract.
StructuredObject writer	Low-copy batch writer where possible; validates row count and column layout.
Result record	Typed result row record for each ResultStream.
Deserializer	Protocol-specific reader that validates metadata before materializing rows.
Contract guard	Startup or call-time guard comparing expected ContractHash with database manifest.
Diagnostics mapping	Maps error envelopes to typed exceptions or result discriminated unions.

Safety rule

The generated client must not invent behavior. It reflects the database contract. A mismatch between generated code and database ContractHash must fail fast before executing a Procedure.

29. SRPL compiler and diagnostics

Compiler stages

Stage	Input	Output	Failure examples
Lexing	SRPL source.	Token stream.	Unknown character, invalid literal, malformed identifier.
Parsing	Token stream.	Concrete syntax tree and AST.	Unexpected token, invalid block, missing end.
Typed binding	AST and CatalogVersion.	Typed AST with names, types, cardinalities.	Unknown object, ambiguous name, invalid type conversion, cardinality violation.
Semantic validation	Typed AST and policies.	Semantic model.	Unbounded loop, forbidden external access, unsafe timestamp, mutation without predicate.
IR lowering	Semantic model.	Relational IR.	Unsupported construct or non-normalizable expression.
Plan candidate generation	Relational IR, catalog, stats.	Candidate logical/physical plans.	No legal access path, policy budget exceeded.
Plan selection	Candidates and evidence.	ProcedurePlan.	No stable plan under resource constraints.
Runtime binding	ProcedurePlan, invocation values.	Executable operator graph.	ContractHash mismatch, invalid StructuredObject layout.

Diagnostic quality

SRPL diagnostics must be compiler-grade, not database-error-grade. A diagnostic should include location, concept, expected shape, actual shape, suggested correction, source contract, and whether the issue is parse-time, bind-time, semantic, policy, or runtime.

Forbidden constructs

Unbounded loops, free recursion, random non-determinism, external network calls, external filesystem calls, dynamic textual SQL, implicit bag semantics, SELECT *, positional output contracts, and ambient NULL-like absence are outside the core language.

30. Storage binary specification backlog

Binary artifacts that need formal specifications

Artifact	Must define
PageHeader	Magic, format version, page type, file ID, page ID, object ID, allocation unit, LSN, epoch, page links, offsets, free bytes, slot count, row count, flags, CRC.
PageTrailer	Payload CRC, page hash, torn-write guards, optional signature hooks.
WalRecord	Record type, LSN, previous LSN, transaction ID, page ID, payload length, checksum, redo payload, undo payload if used.
DatabaseManifest	Database ID, manifest version, snapshot ID, base checkpoint LSN, RequiredWalStartLsn, catalog root, file list hash, previous manifest hash, CRC/signature.
SegmentHeader	Segment ID, allocation unit, compression policy, block count, first/last LSN, hash chain.
BlockHeader	Block ID, compression codec, uncompressed size, payload size, CRC/hash, alignment.
StructuredObjectBatch	Shape ID, row count, column descriptors, layout, absence bitmap if applicable, payload segments.
ResultBatch	ResultStream ID, batch ordinal, row count, column layout, payload length, checksum.

WAL record taxonomy

```

TxBegin
TxCommit
TxRollback
PageAllocate
PageFormat
RowInsert
RowUpdate
RowDelete
IndexInsert
IndexDelete
MvccVersionCreate
MvccVersionClose
MapDeltaAppend
CheckpointBegin
CheckpointEnd
SnapshotBegin
SnapshotEnd
ManifestSwitch
CatalogChangeBegin
CatalogChangeApply
CatalogChangeCommit
SecurityAuditAppend

```

Endianness and canonicalization

Endianness remains open. The final decision must define internal page encoding, WAL encoding, wire encoding, hash canonicalization, cross-platform replay, and mixed x64/ARM64 behavior. The system cannot leave this to host default behavior.

31. Testing, verification, and benchmark discipline

Testing pyramid

Layer	Test types	Required from phase
Types and contracts	Golden tests, schema compatibility tests, descriptor hash tests.	Phase 0.
SRPL compiler	Parser tests, binder tests, semantic diagnostics, IR golden files.	Phase 0.
Transaction kernel	State-machine tests, MVCC visibility tests, rollback tests, poison tests.	Phase 1.
WAL and recovery	Crash tests, torn-write simulation, truncated WAL, manifest corruption, replay idempotence.	Phase 1.
Storage	Page format tests, segment sealing, cold snapshot validation, checksum/hash tests.	Phase 2.
Network protocol	Frame fuzzing, Protobuf compatibility, stream cancellation, backpressure, timeout.	Phase 1.
Security	mTLS tests, permission matrix, audit presence, certificate rotation/revocation.	Phase 5.
Optimizer	Plan golden files, cardinality regression, plan stability, evidence contamination tests.	Phase 3.
HA/DR	WAL shipping tests, replica lag, fencing, promotion, PITR.	After local recovery is stable.

Fuzzing targets

Fuzz the SRPL parser, Protobuf envelope decoding, FrameHeader parsing, WAL record parsing, manifest parsing, page headers/trailers, and StructuredObject batch readers. Any parser touching untrusted or disk-recovered bytes is a fuzz target.

Regression gates

V0 should block release on recovery failure, visibility violation, contract mismatch acceptance, audit omission on security/admin operations, panic in critical paths, silent data corruption, and non-reproducible crash test.

32. Operational profiles and resource governance

Default professional deployment assumption

The baseline assumes controlled professional deployments, often with a small number of application clients, but high RPC frequency and potentially large payloads. The system therefore optimizes for persistent sessions, typed contracts, large batch handling, and precise backpressure rather than uncontrolled public-client scale.

Resource budgets

Budget	Applies to	Examples
Session budget	QUIC sessions and active streams.	Max streams, active RPC count, send/receive buffer limits.
Procedure budget	One Procedure invocation.	CPU time, logical reads, physical reads, WAL bytes, temp bytes, spill bytes.
Transaction budget	Transaction scope.	Version count, lock/latch scope, WAL flush latency, rollback cost.
Storage budget	HotStore and TempStore.	Spool size, temp quota, checkpoint lag, NVMe write pressure.
Optimizer budget	Plan search.	Candidate count, search time, evidence count, plan class quotas.
Map budget	Map maintenance.	Immediate maintenance cost, delta backlog, refresh window, staleness.

Admission control

Admission control must be explicit. A Procedure may be refused before transaction creation if the session, contract, permission, resource budget, or system health state cannot safely accept it. Refusal must produce a typed error and audit where appropriate.

33. Error model and completion semantics

Error families

Family	Examples	Transaction effect
ProtocolError	Invalid frame, unsupported version, bad header CRC, payload length mismatch.	No transaction unless already created; may close stream or session.
AuthenticationError	Invalid certificate, expired certificate, revoked CertificateIdentity.	No transaction.
AuthorizationError	Missing ExecuteProcedure, ReadContract, admin permission, cluster permission.	No transaction.
ContractError	Unknown Procedure, ContractHash mismatch, invalid StructuredObject shape, incompatible ProtocolLayout.	No transaction or rollback if detected late.
SemanticError	SRPL semantic invalidity at definition time.	Catalog change rollback.
ExecutionError	Ensure failure, domain error, operator failure.	Rollback or controlled error depending on Procedure policy.
TransactionError	Conflict, isolation violation, poison, rollback failure.	Rollback or fail-stop.
StorageError	WAL failure, page checksum failure, manifest error.	Fail transaction; may transition database to recovery/forensic mode.
ResourceError	Quota exceeded, temp store pressure, timeout, cancellation.	Cancel/rollback according to state.

Completion envelope

Every invocation must complete with a terminal status: Committed, RolledBack, FailedBeforeTransaction, Cancelled, Poisoned, PermissionDenied, ContractRejected, or SystemUnavailable. RowsAffected is always available for mutation paths where execution reached the mutation stage, even when reporting it adds overhead.

34. Normalized project source transcriptions

This appendix records the main project-source content in normalized American English. It is not a verbatim translation; it is a consolidated transcription of each source normative content.

SRC-01 - Vision, doctrine, and invariants

Andromeda is a relational transactional database system with a native surface of QUIC, custom RPC, Procedure, SRPL, and typed contracts. It rejects SQL ad hoc as the application surface. It is strict at the boundaries and adaptive inside the boundaries. Non-negotiable invariants include Procedure-only execution, typed and hashed Procedure contracts, implicit transactions, durable WAL before visible commit, RAM as non-truth, cold snapshot plus WAL as reconstructible truth, GPU exclusion from commit path, versioned statistics, plan binding to catalog and statistics versions, materialized Maps, typed StructuredObjects, external Modelization only through import, and explainable critical decisions.

SRC-03 - Global architecture

The architecture uses functional engines and transverse planes. It avoids a single parent engine. Core Engine, Network Surface Layer, Execution Engine, SRPL Compiler, Transaction Kernel, Storage Engine, Catalog and Contract Engine, Analytics and Statistics Engine, Predictive Evidence Engine, Internal Analytical Plane, Security/IAM Plane, Administration Plane, Cluster Availability Engine, and Observability/Forensic Plane have separate responsibilities.

SRC-05 - SRPL

SRPL is a strict relational procedure language. Every Procedure is transactional by essence. The engine creates Invocation, TransactionScope, TxId, MVCC visibility, WAL coverage, and Commit/Rollback. Time must be provided through transaction or invocation timestamps, not unsafe ambient time. Mutations are allowed under contract and WAL coverage. ResultStreams are typed and metadata precedes payload. General unbounded loops, free recursion, external network, external filesystem, and non-deterministic random are forbidden.

SRC-08 - Transaction, WAL, MVCC, recovery

The Transaction Kernel owns transaction state. Commit visible equals durable WAL. The canonical state is the last valid Cold Snapshot plus durable WAL after that snapshot. The write pipeline moves from Procedure invocation to RAM modifications, WAL records, durable WAL flush, visible commit, HotStore writeback, checkpoint, and cold snapshot publication. MVCC visibility uses BeginTs and EndTs. Recovery reads manifest, verifies integrity, mounts snapshot, replays WAL, cancels incomplete transactions, rebuilds hot structures, verifies invariants, and opens the database.

SRC-09 and SRC-18 - Storage

Storage is RAM active, NVMe transactional/hot, and HDD cold consolidated. HDD does not receive direct OLTP pressure. Pages have headers and trailers with checksums and torn-write guards. Cold

snapshots are never updated in place. Cold segments are contiguous, append-only, immutable after publication, and validated before manifest switch. Maintenance fences the target, backs up, rebuilds contiguous segments, validates, switches manifest atomically, and cleans up later.

SRC-10 - Network

Andromeda uses QUIC as transport and a strongly typed custom application protocol. Application, Administration, and HA/DR Cluster surfaces are separated. Result metadata, RowCountExact or TotalRowsExact when contractual, column descriptors, batch descriptors, payload batches, and completion are sent in order. Backpressure reacts to buffer saturation, slow clients, execution saturation, WAL lag, NVMe pressure, and temp quota.

SRC-11 - Optimizer, Procedure Store, statistics, evidence

Procedure Store records actual invocation behavior. The optimizer consumes catalog, constraints, statistics, Procedure Store, ScenarioEvidence, hardware profile, storage temperature, and policy. It is cost-based, evidence-based, workload-aware, stability-aware, and bounded. Predictive Evidence produces evidence only. Statistics produce cardinality, distribution, histograms, skew, density, selectivity, and storage temperature, with optional GPU compute and controlled publishing.

SRC-12 and SRC-13 - Security and HA/DR

Security follows mTLS to CertificateIdentity to UserPrincipal to roles/groups to permissions to policies to audit. Administration is powerful and strictly audited. Debug uses isolated snapshots and never mutates active production. HA/DR V0 is Single Primary plus Replicas with WAL shipping, snapshot shipping, quorum, fencing, promotion to a consistent LSN, explicit RPO/RTO, backups, PITR, and forensic startup.

SRC-14 and SRC-16 - Hardware and roadmap

Rust is the primary language. Unsafe Rust is expected but must be isolated, documented, tested, audited, and wrapped in safe APIs. CPU capabilities are detected at runtime. GPU is for analytics and evidence, not commit or recovery. NVMe is the transactional hot layer. HDD is cold truth. The implementation starts with Catalog, minimal SRPL, Transaction Kernel, WAL, Recovery, and contractual RPC.

35. Appendix A - Decision register

Decision	Title	Statement	Status
DEC-001	Native surface	Andromeda exposes no SQL ad hoc application surface. The only behavioral unit is a cataloged Procedure invoked through typed RPC over QUIC.	Locked
DEC-002	Protocol stack	QUIC remains the transport. The Andromeda application protocol remains custom. Protobuf becomes the canonical wire contract format for command, contract, metadata, and selected payload messages. gRPC is explicitly excluded.	Updated and locked
DEC-003	Contract exposure	The engine publishes Procedure contracts, input shapes, structured object shapes, output shapes, policies, versions, and ContractHash values for client validation and code generation.	Locked
DEC-004	Transaction scope	Every Procedure is transactional by design. A caller does not opt into a transaction; the engine creates an Invocation, TransactionScope, TxId, visibility rule, WAL coverage, and completion state.	Locked
DEC-005	Durability invariant	A commit becomes visible only after durable WAL coverage. The reconstructible truth is the last valid cold snapshot plus durable WAL since that snapshot.	Locked
DEC-006	Hot/cold storage	RAM is volatile working state. NVMe/SSD is the transactional hot layer. HDD is the validated cold truth layer for snapshots and archives. Cold storage is append-only and immutable after publication.	Locked
DEC-007	GPU role	GPU acceleration is allowed for analytics, statistics, evidence, and large scans outside the commit path. GPU is not allowed in commit, rollback, WAL durability, recovery, or short MVCC visibility checks.	Locked
DEC-008	Maps	Maps are materialized projections with explicit coherence policies. There is no generic virtual View object in the native vocabulary.	Locked
DEC-009	Predictive evidence	Benchmark and simulation engines produce evidence only. The optimizer decides under policy and must remain bounded, explainable, versioned, and observable.	Locked
DEC-010	V0 implementation priority	The first vertical slice is Catalog + minimal SRPL + Transaction Kernel + WAL + Recovery + contractual RPC. GPU, advanced benchmark, and OLAP are later phases.	Locked

36. Appendix B - Risk register

Risk	Title	Severity	Mitigation
RISK-001	Scope explosion	Very high	Maintain a strict vertical prototype and defer GPU, advanced HA/DR, and complex analytics until the kernel is recoverable.
RISK-002	Custom QUIC protocol complexity	High	Keep the V0 envelope small, define explicit stream taxonomy, and benchmark Quinn, s2n-quic, and MsQuic before committing.
RISK-003	Protobuf schema drift	High	Use stable packages, reserved tags, reserved names, explicit presence, semantic versioning, and compatibility tests.
RISK-004	Unsafe Rust defects	Very high	Encapsulate unsafe code, require safe public APIs, fuzz tests, property tests, Miri/sanitizer campaigns where applicable, and code ownership.
RISK-005	Recovery under-tested	Critical	Run crash tests from Phase 1 and treat recovery bugs as release-blocking.
RISK-006	Plan instability	High	Use bounded plan classes, hysteresis, plan evidence scores, and controlled invalidation on CatalogVersion or StatsVersion changes.
RISK-007	Map maintenance cost	High	Require explicit refresh policy, complexity score, maintenance budget, and criticality classification.
RISK-008	Statistics overconfidence	High	Separate confidence, criticality, freshness, realism, and stability scores; never let predictive evidence force a plan alone.
RISK-009	Security bypass through surfaces	Critical	Keep Application, Administration, and HA/DR surfaces separate with different certificates, endpoints, permissions, quotas, and audits.

37. Appendix C - Source-to-section traceability

This table records where the core sources are integrated. A later version can add line-level evidence references if the source repository stores stable line numbers.

Source	Integrated sections
SRC-00	Executive summary; system overview; source ledger; roadmap.
SRC-01	Design doctrine; invariants; acceptance gate; decision register.
SRC-02	Vocabulary throughout; Appendix E glossary.
SRC-03	System overview; functional engine architecture; dependency prohibitions.
SRC-04	Catalog and metadata architecture.
SRC-05	SRPL language model; transaction kernel; Procedure contract design.
SRC-06	Type system and data model; StructuredObject protocol policy.
SRC-07	Maps and internal analytics.
SRC-08	Transaction kernel; recovery and forensic startup.
SRC-09	Storage architecture.
SRC-10	QUIC plus Protobuf without gRPC; Procedure contract design.
SRC-11	Procedure Store, optimizer, statistics, and evidence.
SRC-12	Security, IAM, administration, and audit.
SRC-13	HA/DR, backup, restore, and forensic operations.
SRC-14	Rust implementation baseline; hardware acceleration boundaries.
SRC-15	Catalog and metadata architecture; Modelization and import.
SRC-16	V0 roadmap; risk register; open decisions.
SRC-17	External research foundation; bibliography.
SRC-18	Storage architecture; ColdStore contiguity.
SRC-19	External research foundation; transaction, algebra, statistics, and index synthesis.
SRC-20	Bibliography and research classification.
SRC-21	SRPL language model; SQL critique; compilation and plan cache.
CHAT-CTX	Updated Protobuf direction; C# source generator use; QUIC persistent connection assumptions; RowCountExact; multi-result streams.

38. Appendix D - External reference ledger

Tag	Reference	URL
EXT-QUIC-9000	RFC 9000 - QUIC: A UDP-Based Multiplexed and Secure Transport	https://www.rfc-editor.org/rfc/rfc9000.html
EXT-QUIC-9001	RFC 9001 - Using TLS to Secure QUIC	https://www.rfc-editor.org/rfc/rfc9001.html
EXT-QUIC-9002	RFC 9002 - QUIC Loss Detection and Congestion Control	https://www.rfc-editor.org/rfc/rfc9002.html
EXT-QUIC-9221	RFC 9221 - Unreliable Datagram Extension to QUIC	https://www.rfc-editor.org/rfc/rfc9221.html
EXT-PROTOBUF-ED2024	Protocol Buffers Edition 2024 language specification	https://protobuf.dev/reference/protobuf/edition-2024-spec/
EXT-PROTOBUF-BP	Protocol Buffers best practices	https://protobuf.dev/best-practices/dos-donts/
EXT-RUST-2024	Rust Edition Guide - Rust 2024	https://doc.rust-lang.org/edition-guide/rust-2024/
EXT-TOKIO	Tokio runtime documentation	https://docs.rs/tokio/latest/tokio/runtime/
EXT-TRACING	tracing crate documentation	https://docs.rs/tracing/
EXT-QUINN	Quinn QUIC implementation documentation	https://docs.rs/quinn/
EXT-S2N	s2n-quic documentation	https://docs.rs/s2n-quic/
EXT-MSQUIC	MsQuic documentation	https://microsoft.github.io/msquic/
EXT-CODD	Edgar F. Codd - A relational model of data for large shared data banks	https://dl.acm.org/doi/10.1145/362384.362685
EXT-ARIES	C. Mohan et al. - ARIES recovery method	https://research.ibm.com/publications/aries-a-transaction-recovery-method-supporting-fine-granularity-locking-and-partial-rollbacks-using-write-ahead-logging
EXT-VOLCANO	Graefe and McKenna - The Volcano optimizer generator	https://dl.acm.org/doi/10.5555/645478.757691
EXT-BWTREE	Microsoft Research - The Bw-tree	https://www.microsoft.com/en-us/research/publication/the-bw-tree-a-b-tree-for-new-hardware/
EXT-FASTER	Microsoft Research - FASTER key-value store	https://www.microsoft.com/en-us/research/publication/faster-concurrent-key-value-store-place-updates/
EXT-MS-STYLE	Microsoft Style Guide - headings and scannable content	https://learn.microsoft.com/en-us/style-guide/scannable-content/headings

39. Appendix E - Canonical glossary

Term	Canonical meaning
Instance	An Andromeda process/engine with its QUIC surfaces, system database, and user databases.
System Database	Internal master database containing registries, policies, security, catalog history, hardware profiles, and HA/DR state.
Database	User database isolated under the instance.
Namespace	Logical naming boundary for tables, maps, enums, structured objects, and procedures. Equivalent pedagogically to SQL schema.
Table	Typed stored relation with columns, rows, constraints, access paths, and statistics.
Map	Materialized projection with storage, statistics, contract, dependencies, and coherence policy.
Enum	Cataloged named discrete type; may support flags mode.
StructuredObject	Typed tabular shape used as Procedure parameter or result shape.
Procedure	Only callable behavioral unit; typed, contract-bound, transaction-bound, observable, and invoked by RPC.
SRPL	Strict Relational Procedure Language.
ContractHash	Canonical hash of a stable contract representation.
CatalogVersion	Version of the catalog after a transactional mutation.
StatsVersion	Published version of statistics used by planning.
Procedure Store	Historical store of Procedure invocations, plan classes, resource usage, and outcomes.
ScenarioEvidence	Bounded evidence generated by benchmark/simulation for optimizer consumption.
ResultStream	Typed result stream with metadata before payload.
RowCountExact	Exact row count advertised before payload when required by contract.
HotStore	NVMe/SSD storage for recent transactional state, pages, WAL, versions, and temp.
ColdStore	HDD storage for validated immutable snapshots, manifests, archives, and cold indexes.
Modelization	External design artifact that becomes real only through DefinitionBatch import.
DefinitionBatch	Ordered, validated, auditable batch of catalog definitions applied transactionally.

40. Appendix F - Protobuf contract baseline

Package layout

```
syntax = "editions";
edition = "2024";

package andromeda.protocol.v1;

message ProcedureManifest { ... }
message ProcedureContract { ... }
message RpcExecuteRequest { ... }
message RpcMetadata { ... }
message RpcBatch { ... }
message RpcCompletion { ... }
message ErrorEnvelope { ... }
```

Compatibility rules

- Do not reuse field numbers.
- Reserve deleted field numbers and names.
- Prefer additive changes.
- Keep required semantic fields in stable envelope messages, not scattered in payload-specific messages.
- Use explicit presence semantics where the toolchain supports it.
- Keep Protobuf JSON mapping for diagnostics and debugging only.
- Generate compatibility tests from descriptor sets.
- Bind descriptor set hash into ProtocolLayout and ContractHash computation.

Boundary rule

Generated Protobuf types are input/output boundary types. The engine converts them to internal Rust types before catalog, optimizer, transaction, or storage logic uses them.

41. Appendix G - Minimal V0 Protobuf envelope sketch

```
message FrameEnvelope {
  uint32 protocol_major = 1;
  uint32 protocol_minor = 2;
  bytes contract_hash = 3;
  uint64 catalog_version = 4;
  uint64 request_id = 5;
  uint64 session_id = 6;
  optional uint64 tx_id = 7;
  PayloadKind payload_kind = 8;
  bytes payload = 9;

  reserved 10 to 19;
}

enum PayloadKind {
  PAYLOAD_KIND_UNSPECIFIED = 0;
  PAYLOAD_KIND_HELLO = 1;
  PAYLOAD_KIND_AUTH = 2;
  PAYLOAD_KIND_CONTRACT_REQUEST = 3;
  PAYLOAD_KIND_CONTRACT_RESPONSE = 4;
  PAYLOAD_KIND_RPC_EXECUTE_REQUEST = 5;
  PAYLOAD_KIND_RPC_METADATA = 6;
  PAYLOAD_KIND_RPC_BATCH = 7;
  PAYLOAD_KIND_RPC_COMPLETION = 8;
  PAYLOAD_KIND_ERROR = 9;
}

message StructuredObjectHeader {
  string name = 1;
  bytes contract_hash = 2;
  uint64 row_count_exact = 3;
  uint32 column_count = 4;
  StructuredObjectLayout layout = 5;
  uint64 payload_length = 6;

  reserved 7 to 15;
}
```

42. Appendix H - Minimal SRPL grammar direction

```
procedure_declaration
  = "procedure" qualified_name accepts_clause returns_clause? procedure_body ;

accepts_clause
  = "accepts" parameter_list ;

returns_clause
  = "returns" result_contract_list ;

procedure_body
  = "begin" statement* "end" ";" ;

statement
  = ensure_statement
  | let_statement
  | mutation_statement
  | return_statement
  | try_catch_statement
  | bounded_loop_statement ;

mutation_statement
  = insert_statement | update_statement | delete_statement | merge_statement ;

bounded_loop_statement
  = "for each" identifier "in" finite_collection "do" statement* "end for" ";"
  | "repeat" integer_literal "times" "do" statement* "end repeat" ";" ;

forbidden_core_construct
  = unbounded_while | free_recursion | external_network | external_filesystem |
  nondeterministic_random ;
```